

EE671: VLSI Design

Course Project 2

Authors

Advay Bapat (23B1225)

Dinesh Methre (23B1228)

Pranav Malpani (23B1279)

Suketu Patni (23B1299)

November 29, 2025

Contents

1	Introduction	2
2	Literature Review	2
3	Verilog Design	2
3.1	Memory Layout	2
3.2	ALU state machine	3
3.3	Addition/Subtraction	4
3.4	Inverse	6
3.5	Multiplication	6
3.6	Systolic Matrix Multiplier	6
3.6.1	Systolic Entry Module	6
3.6.2	Matrix Multiplier	7
3.6.3	Process Elements	7
3.7	I/O	7
3.8	GTKWave forms	8
4	Synthesis	12
4.1	GTKWave forms	16
5	Physical Design	20
5.1	GTKWave forms	22
5.2	Table of Reports	28
6	Instructions to Replicate	28
7	Challenges faced	28
8	Contributions	30

1. Introduction

We have in this project implemented an 8-bit Integer Matrix Arithmetical Logical Unit (ALU), equipped with addition, subtraction, transpose, inverse and matrix multiplication functionality for matrices upto 3×3 in order.

The design features an internal memory for random access and storing intermediate results. In our view, this design should be more efficient than instantiating different registers at each stage of the computation, which would significantly increase the size and might create problems during physical design.

The device also supports a single bus for loading in the data and reading the result. This was created so that the testbench could interact with the design and RTL and make testing much simpler. Strictly speaking, this I/O is not required if the ALU is to be integrated directly on a larger chip.

We think the matrix multiplier and inverse features in our design could replace the higher-level implementations of these algorithms, due to being specifically designed for these operations and as a result being parallelized.

2. Literature Review

Systolic arrays have a very interesting and shrouded history, being first used in WW2 to crack German ciphers. Because of confidentiality, the credit for their discovery is often given to H.T. Kung and C.E. Leiserson in their seminal 1978 paper, "*Systolic Arrays (for VLSI)*". The proposal was of a network of identical processing elements (PEs) designed to rhythmically compute and pass data through the system. The primary motivation was to maximize data locality. In this architecture, once a data item is fetched from memory, it traverses through multiple PEs, being reused for multiple operations before returning to memory, thereby significantly reducing memory access energy and bandwidth requirements.

The most common implementation as in today is characterized by a 2D grid of systolic elements, each of which has a multiplier and an accumulator, and is capable of accepting two inputs and giving two outputs. Their specific working is discussed in detail in the paper. Their original application was for heavy and dense linear algebra applications, and they are still used for it today, but now they have multiple other uses as well, owing to the AI boom. Perhaps their most pertinent uses is in NPU's (Neural Processing Units) and TPU's (Tensor Processing Units), which form the backbone of modern AI computations. But apart from such commercial chips which are application specific, most "hobby-project" systolic array implementations (or those in academia) are done on FPGAs and not ASICs owing to the high amount of multipliers required. So a physical design of systolic arrays is quite novel.

3. Verilog Design

The following was the structure that we followed while making the design in verilog.

3.1. Memory Layout

Note, the ALU has 39 bytes of memory (with indices ranging from 36 to -2), structured as follows:

- **mem[36]:** Stores the row and column lengths of A and B.
 - **7-6:** Rows of A.
 - **5-4:** Rows of B.
 - **3-2:** Columns of A.

– **1-0:** Columns of B.

- **mem[35] - mem[27]:** These are the (row-wise) entries of the matrix A. We have used fixed addressing for each possible (row index, col. index) pair, i.e. A_{11} is always at mem[35], A_{12} is always at mem[34], A_{31} at mem[29], etc. Whenever the order of the matrix is other than 3×3 , we still assume it to be 3×3 , only with appropriate 0-padding.
- **mem[26] - mem[18]:** These are the (row-wise) entries of the matrix B. Similar to A, B_{11} is always at mem[26], B_{12} is always at mem[25], B_{31} at mem[20], etc. The same 0-padding rule is followed.
- **mem[17] - mem[0]:** These are the (row-wise) entries (results) of the matrix C, corresponding to the opcode given. The same rule as above matrices is applied, the only difference being that the entries of C can now take up 16 bits instead of 8 (due to multiplication being an available operation). So, C_{11} corresponds to {mem[17], mem[16]}, and so on.
- **mem[-1] - mem[-2]:** This is the place where the determinant of the matrix A is stored, in the case of the opcode being `inv`.

3.2. ALU state machine

After some trial and error, we landed on the following state machine (Displayed in Fig. 1).

Init

The initial state is `s0`. Upon receiving the reset signal, the finite state machine (FSM) goes to `s0`. The FSM leaves this state only upon receiving an `enable` pulse and a valid opcode. The next state is decided through the opcode received.

Order Checking States

If the opcode corresponds to addition, subtraction, multiplication, or inversion, the FSM transitions to a specific order-checking state to validate the matrix dimensions stored in mem[36].

- **Addition/Subtraction (`s_addsub_order_check`):** The FSM checks if the row counts match (`mem[36][7:6] == mem[36][5:4]`) and if the column counts match (`mem[36][3:2] == mem[36][1:0]`). If valid, it transitions to `s_add` or `s_sub`, otherwise, it raises `ord_err` and returns to `s0`.
- **Multiplication (`s_mul_order_check`):** The FSM checks if the columns of Matrix A equal the rows of Matrix B (`mem[36][3:2] == mem[36][5:4]`). If valid, it transitions to `s_mul` otherwise, it raises `ord_err` and returns to `s0`.
- **Inversion (`s_inv_order_check`):** The FSM checks if the input is a square matrix by comparing rows to columns (`mem[36][7:6] == mem[36][3:2]`). If valid, it transitions to `s_inv3`, otherwise, it raises `ord_err` and returns to `s0`.

Calculation States

For operations requiring element-wise processing (Mul) or sequential calculation (Add, Sub, Transpose, Inverse), the FSM uses an internal counter `index_count` ranging from 0 to 9.

- **Add/Sub/Transpose (`s_add`, `s_sub`, `s_t`):** In these states, `mux_en` is set high. The ALU computes the result for the current index of C, and writes it to the resulting location in memory. The counter increments on every clock cycle. Thus, the computation is serial, and uses only one adder. When `index_count` reaches 9 (all of the entries of C are computed), `mux_en` is disabled, and the FSM transitions to `s_done_pulse`.

- **Inverse (s_{inv3}):** This state calculates cofactors and the determinant. mux_en is high to write cofactor results. During indices 0, 1, and 2, the determinant of the Minor matrices is accumulated in det_accum using column 0 expansion. When $index_count$ reaches 9, the calculated determinant is written to $mem[-1]$ and $mem[-2]$. Simultaneously at count 9, if det_accum is 0, the $singular_err$ flag is raised. The FSM then transitions to s_done_pulse .

Multiplication Sequence

Matrix multiplication is handled by an external systolic array and requires this sequence of states in the following order:

1. **s_mul :** The FSM asserts mul_start for one cycle to initialize the systolic array and immediately transitions to s_mul2 .
2. **s_mul2 :** The FSM waits in this state until the mul_done signal is received from the systolic array, indicating calculation completion.
3. **s_mul_write :** Upon completion, the FSM asserts mux_en to write the aggregate results from the systolic array into memory. It then transitions to s_done_pulse .

Completion and Streaming

- **s_done_pulse :** This state asserts the $done_final$ signal for a single clock cycle, indicating that the ALU operation is complete and memory is updated. It transitions unconditionally to s_stream .
- **s_stream :** The FSM streams the results back to the testbench. It asserts $result_valid$. It reads data from memory using $read_ptr$, which decrements from 17 down to -2 (covering the Result Matrix and the Determinant). Once $read_ptr$ reaches -2, the streaming is complete, and the FSM resets to the idle state $s0$.

3.3. Addition/Subtraction

The Adder/Subtractor implemented is a standard 16-bit Kogge Stone Adder with the C_{in} now acting as a control bit for subtraction as well as the carry. The input to the Kogge Stone Adder is either 8-bit inputs that are extended internally or 16-bit inputs with the output being 16-bit.

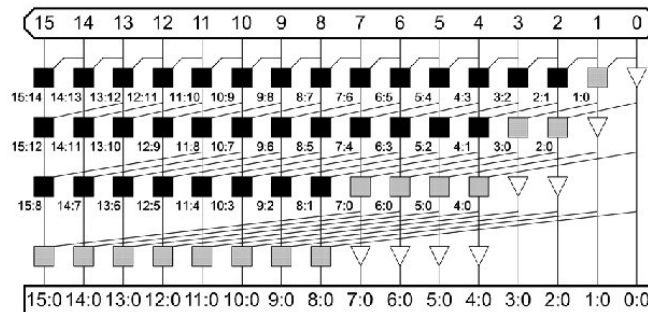


Figure 2: Kogge Stone Adder for 16-bit Inputs.

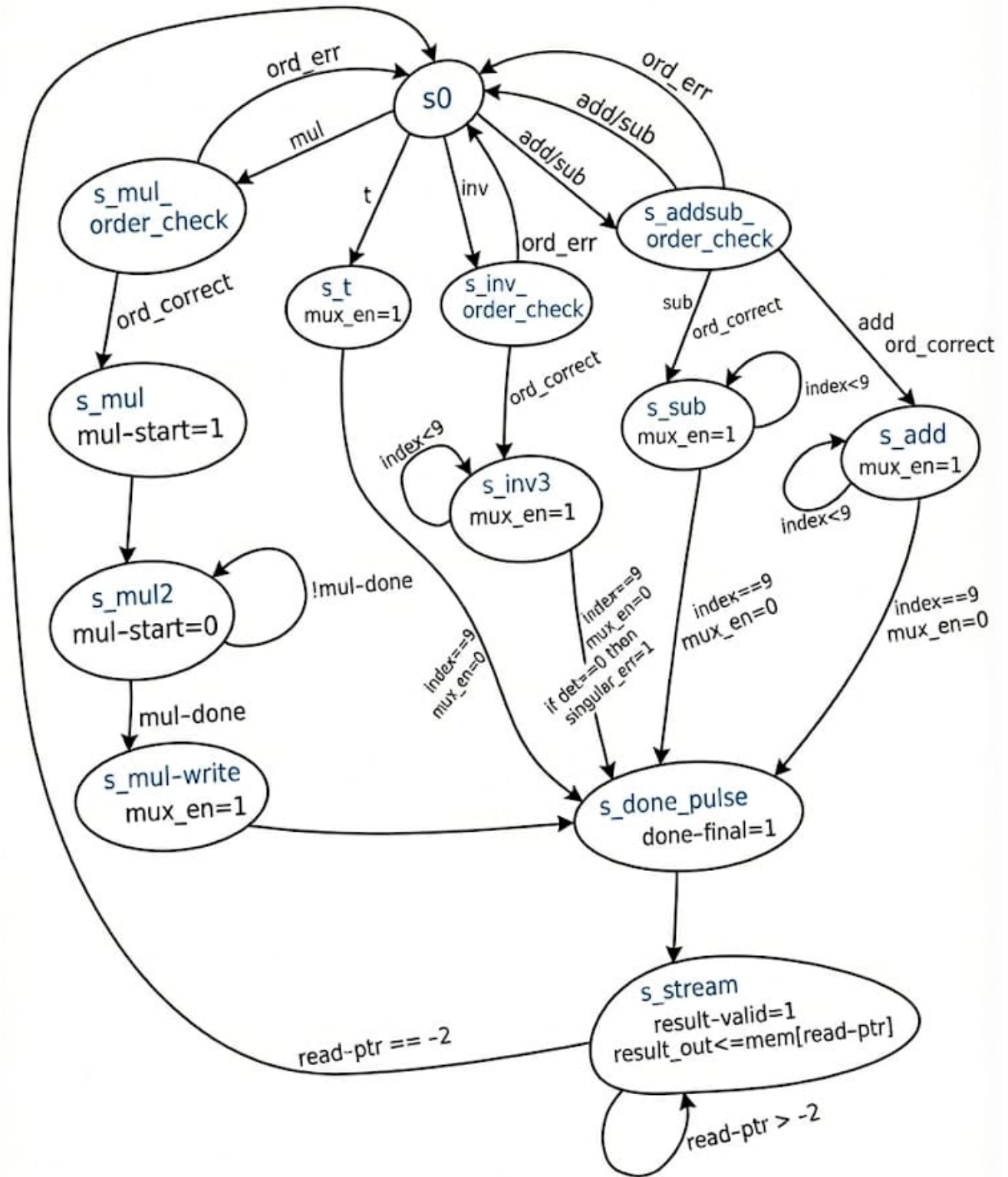


Figure 1: State transition diagram for the ALU.

3.4. Inverse

Apart from transpose, this is the only other unary operation we have implemented. Since for a matrix to even be considered invertible, the number of rows must be equal to the number of columns, the operation starts, as with others, by first checking if its inputs satisfy this condition, in the state `s_inv_order_check`. Once this check has been cleared, it (like the addition, subtraction and transpose states) goes through 9 passes of incrementation of the `index_count`, and for each pass, a combinational block determines the inputs to give the `det_2x2` instance, which correspond to the 2x2 submatrix made by removing the row and column of an element at that `target_idx`. `target_idx` is a 4-bit wire that keeps track of the `index_count` mirrored across the matrix diagonal. We used simple K-maps to determine a cheap combinational logic to find `target_idx` from `index_count`. `det_minor` (the raw value of the determinant) and consequently `inv_result` (the actual cofactor i.e. adjusted for sign) are two 16 bit wires driven by the output of the 2x2 determinant module. It is easily seen that an easy way to determine the required sign is to look at the least significant bit of the `index_count`. Lastly, as in the other operations, it is written to the 144-bit wire `data_src_3` which is then muxed with the other operations and written to memory. It is important to note that, because we follow a fixed memory addressing system, we are always taking inverses of 3x3 matrices. In case 2x2 or 1x1 matrices are input, they must be padded into a 3x3 matrix by putting '1's on all the remaining diagonal entries and '0's everywhere else.

3.5. Multiplication

The Multiplication is done by first computing the partial products using modified booth encoding and then using the Wallace Multiplier implementation to reduce them. The standard implementation of the Booth-Wallace multiplier for an 8-bit $\times$ 8-bit multiplication is used.

3.6. Systolic Matrix Multiplier

To perform the matrix multiplication part of this project, we decided to use systolic arrays rather than using the standard way of doing the matrix multiplication. This leads to a much faster implementation due to parallel computation.

The systolic array is a network of data processing units (DPUs) computing partial products and using from the data it receives each clock cycle from upstream and passes it downstream. The data received is a spatially skewed version of the input matrices A and B, shifting each row of A by one more than the previous row/column and shifting each column of B by one more than the previous row/column and padding it with a dummy input zero(0). An example of the operation for two 2x2 matrices is shown below.

3.6.1 Systolic Entry Module

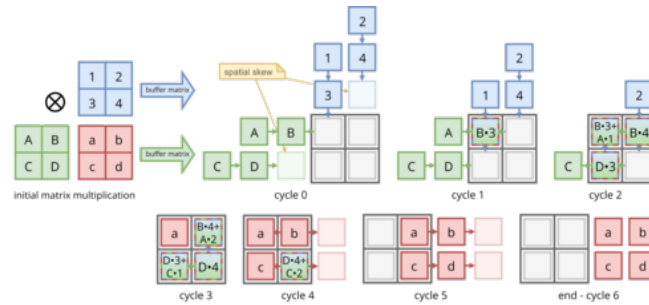


Figure 3: Using Systolic Arrays for Matrix Multiplier.

As seen in Fig. 3, and in standard systolic array implementations, the input has to be temporally skewed. The skew increases as we go down the rows and across the columns. To not overcomplicate the ALU, we made a new module which latches on all the three entries together and handles the skewing and the filling in of the values internally, called the `entry_module`.

3.6.2 Matrix Multiplier

This is the main controller module which takes the inputs in from the systolic entry modules, and controls 9 process elements. The main task of this module is to synchronize the data transfer between process elements. It also controls the final outputs given by process elements.

3.6.3 Process Elements

This is the main workhorse of the matrix multiplier. Each Process Element houses a combinational adder and a multiplier, the same as described above. It also has an accumulator to add the elements of the dot product between rows of A and columns of B. Additionally, it performs the function of transferring the inputs (labelled `left` and `up` it gets) to the next process element via outputs (labelled `right` and `down`). It also keeps track of the number of computations that it is expected to do (based on the input given by matrix multiplier), and raises the `done` flag, when it is done with the specified number of steps.

3.7. I/O

As mentioned earlier, we have chosen to have an internal memory. This memory can be written to and be read out of, by the testbench serially. The write to memory happens before sending the `enable` pulse, and the read happens after receiving the `result_valid` signal, when the ALU has completed computation and reached the `s_stream` state.

Our main focus was optimizing the speed of computation and the size of ALU and not exactly the time to setup the ALU for computation, as we believe this can be easily bypassed if this ALU is integrated directly on a larger chip. Also, we found that the synthesis tool does not synthesize the memory unless something is driving it, which led us to use this method, where the testbench actually explicitly interacts with it through pins.

3.8. GTKWave forms

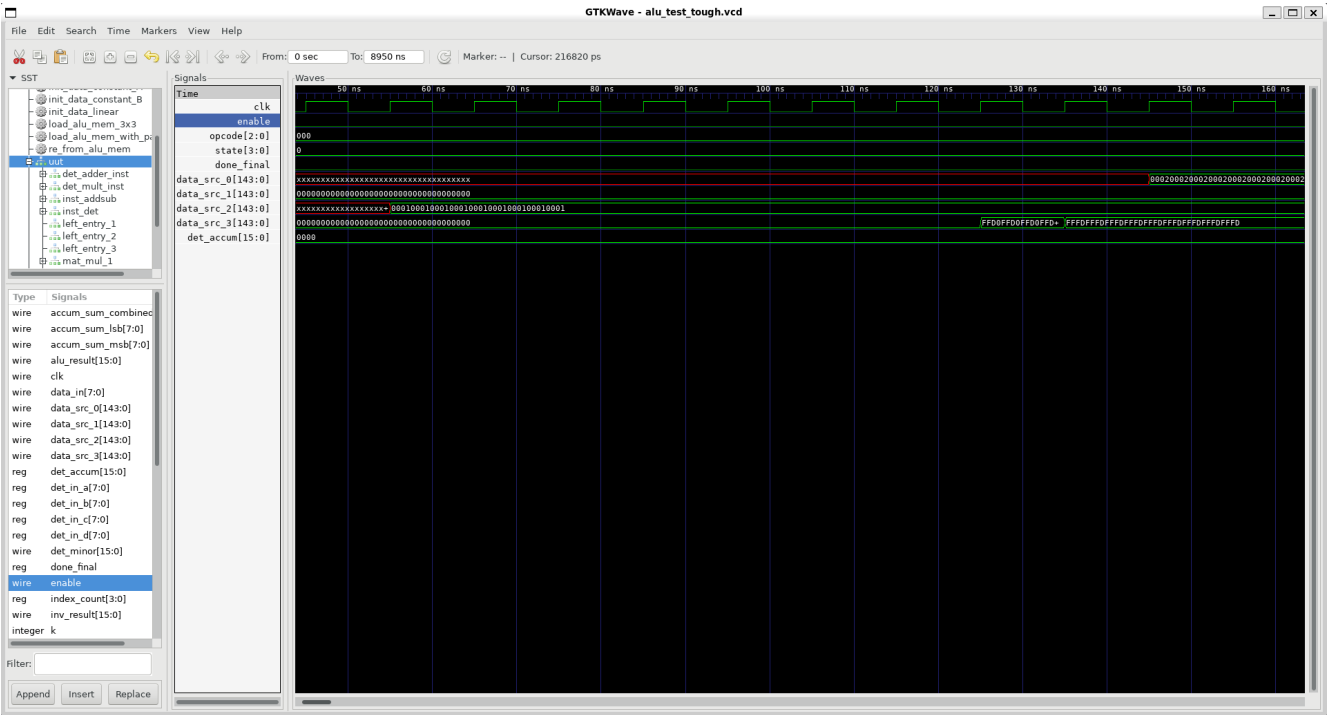


Figure 4: Taking Inputs

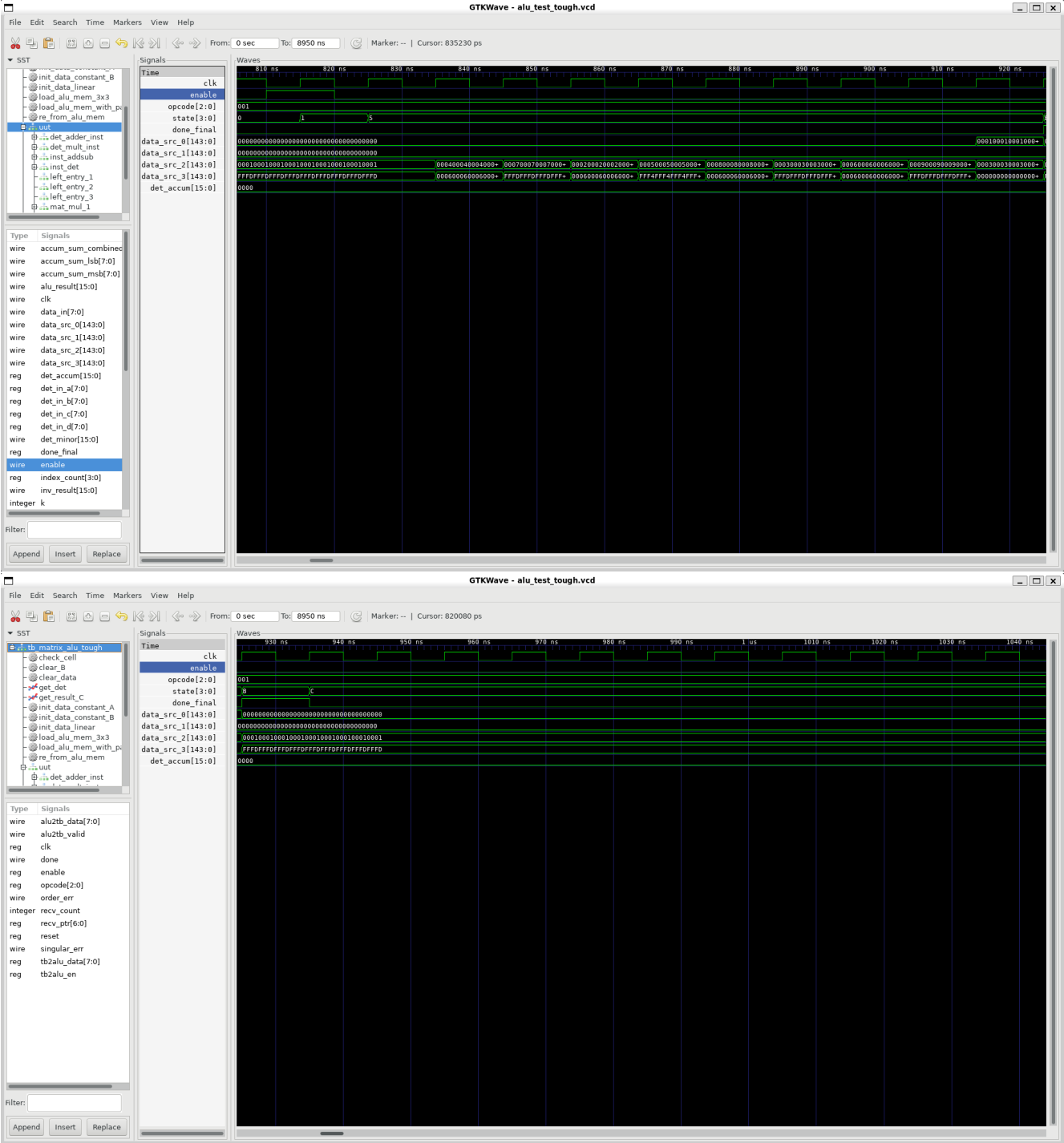


Figure 6: Sub - Enable and Output

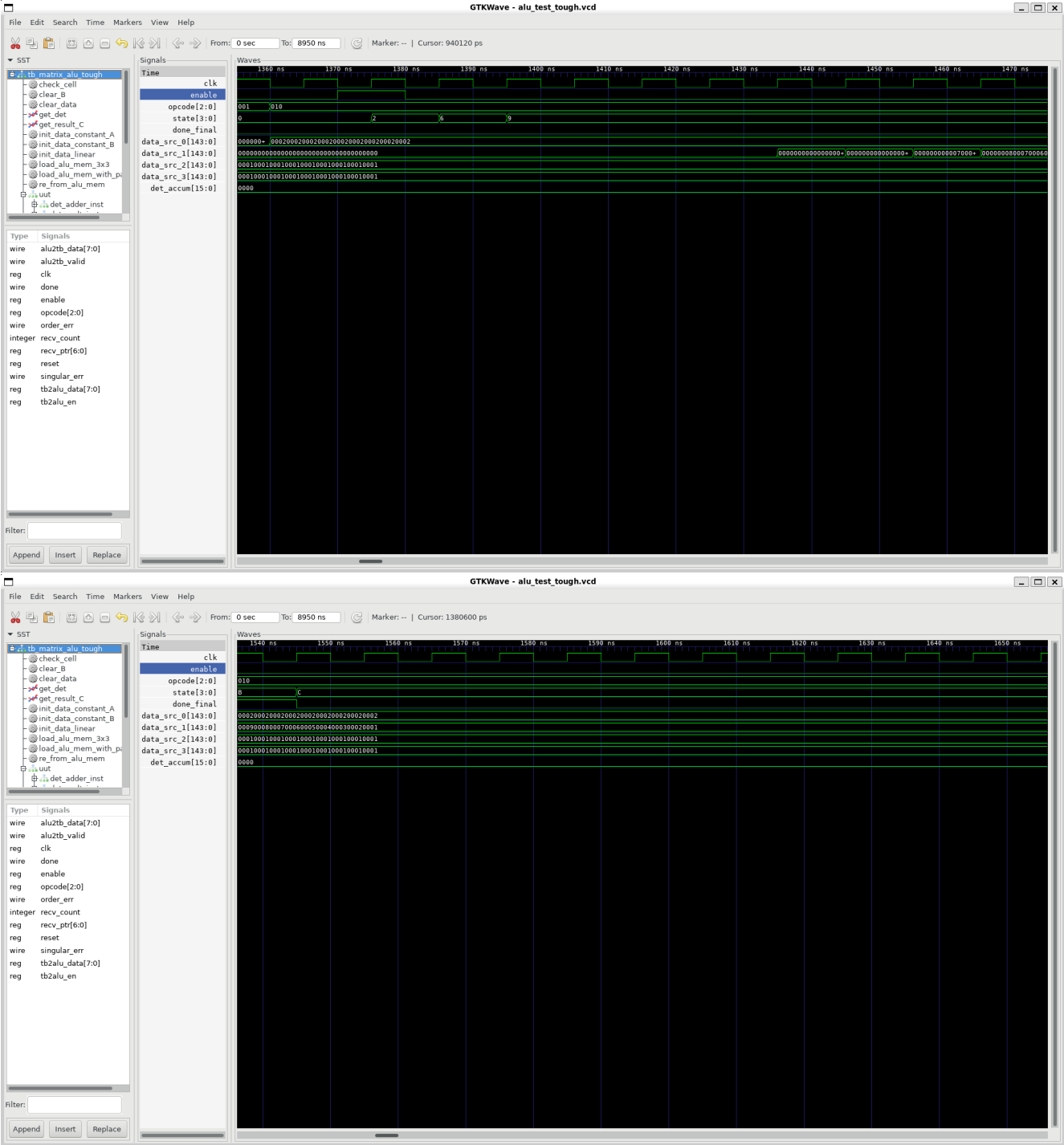


Figure 7: Mul - Enable and Output

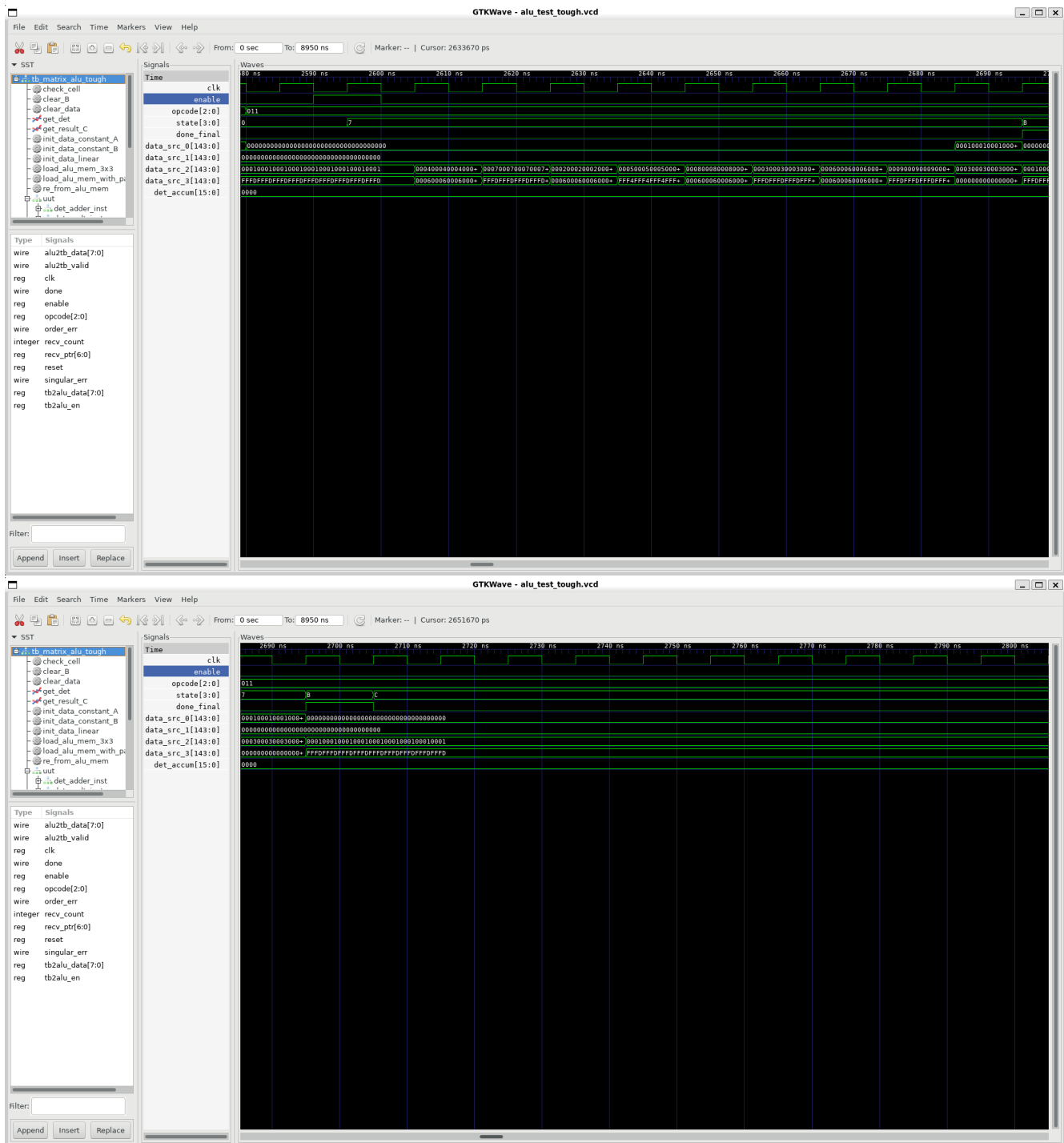


Figure 8: Transpose - Enable and Output

4. Synthesis

Now that we have the ALU fully written and tested with the testbench, now we can move to the Synthesis. Due to the premade scripts, we adjusted the code by setting the FSM (Finite State Machine) as the Top level and include every other file in it and therefore, there was no need to change the basename as well.

After editing the constraints.sdc file to change the clock to 100MHz and adding the pins. The output of each of the reports generated are shown below

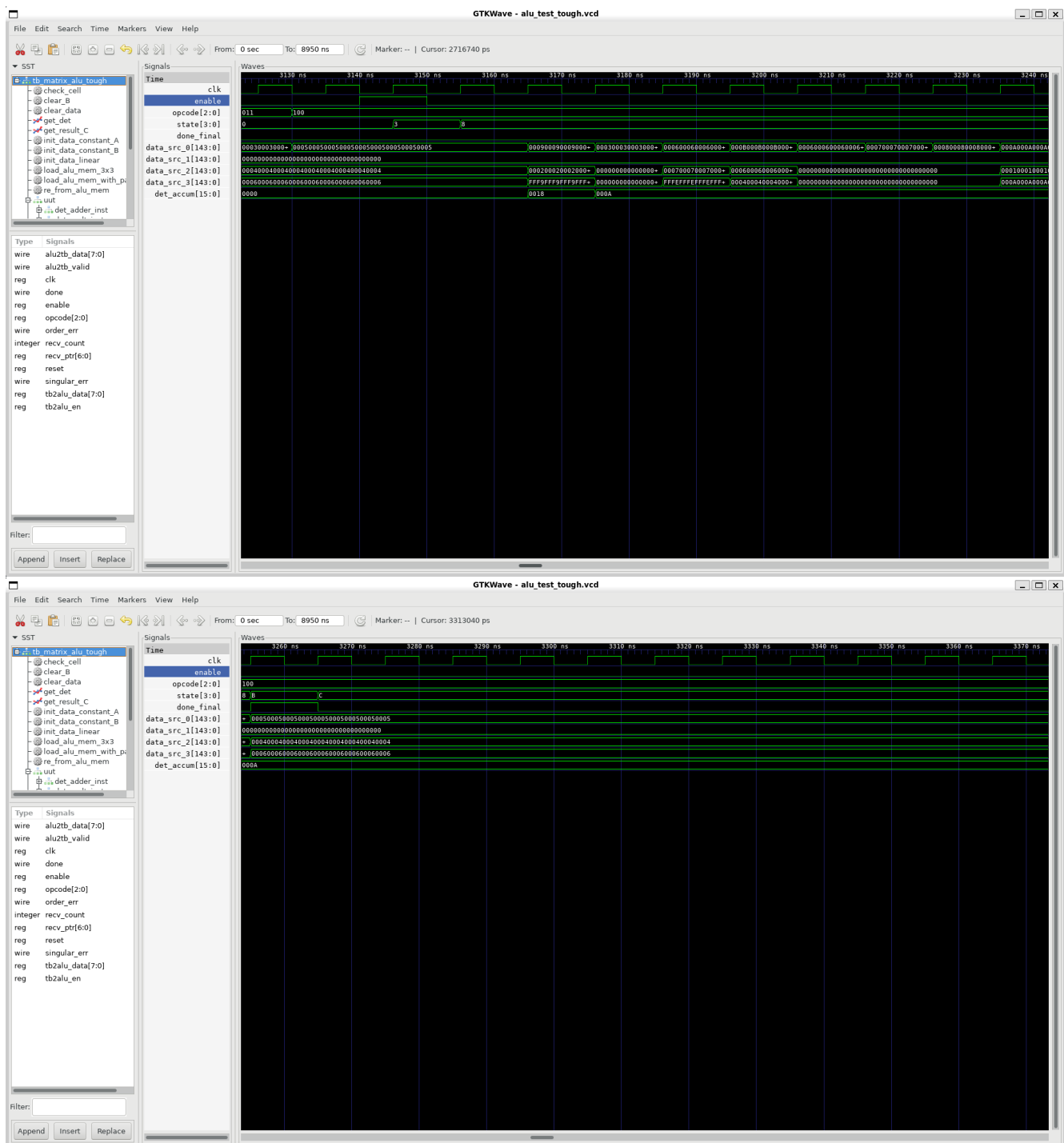


Figure 9: Inverse - Enable and Output

```

EE671_36@vlsi73:~/PnR_Exam X + v
g161358/C0 FA1D0 2 4.8 45 +119 2873 F
g84558__4733/A1 2873
g84558__4733/Z A022D0 2 4.8 45 +76 2949 F
g84550__5122/A1 2949
g84550__5122/Z A022D0 1 2.7 33 +66 3016 F
g161357/CI 3016
g161357/C0 FA1D0 2 4.8 45 +76 3092 F
g84529__9945/A1 3092
g84529__9945/Z A022D0 2 4.8 45 +76 3168 F
g84516__4733/A1 3168
g84516__4733/Z A022D0 2 4.7 44 +76 3243 F
g84512__1881/B2 3243
g84512__1881/Z A022D0 2 5.0 46 +87 3330 F
g84508__8246/B1 3330
g84508__8246/ZN A0I22D1 1 2.9 64 +55 3386 R
g84505__1705/A3 3386
g84505__1705/Z XOR3D1 1 3.3 31 +74 3459 R
g2/A4 3459
g2/Z XOR4D1 1 2.5 46 +105 3564 F
g84503__1617/A1 3564
g84503__1617/Z A022D0 1 2.7 38 +67 3631 F
det_accum_reg[15]/D <<< DFQD1 3631
det_accum_reg[15]/CP setup 0 +12 3644 R
-----
(clock clk) capture 10000 R
uncertainty -100 9900 R
-----
Cost Group : 'clk' (path_group 'clk')
Timing slack : 6256ps
Start-point : index_count_reg[0]/CP
End-point : det_accum_reg[15]/D
[EE671_36@vlsi73 Synthesis]$ |

```

Figure 10: Output of the timing.rpt

```

EE671_36@vlsi73:~/PnR_Exam X + v
[EE671_36@vlsi73 Synthesis]$ cat area.rpt
=====
Generated by: Genus(TM) Synthesis Solution 21.19-s055_1
Generated on: Nov 29 2025 04:11:39 am
Module: alu
Technology libraries: tcbn65gplustc_ccs 200
physical_cells
Operating conditions: NCCOM
Interconnect mode: global
Area mode: physical library
=====
Instance Module Cell Count Cell Area Net Area Total Area
-----
alu 5237 22098.600 10356.063 32454.663
[EE671_36@vlsi73 Synthesis]$ |

```

Figure 11: Output of the area.rpt

```

EE671_36@vlsi73:~/PnR_Exam X + | v
OAI31D1      4    10.080  tcbn65gplustc_ccs
OAI32D1      6    17.280  tcbn65gplustc_ccs
OAI33D1      1     3.240  tcbn65gplustc_ccs
OR2D1       21    45.360  tcbn65gplustc_ccs
OR2XD1      14    30.240  tcbn65gplustc_ccs
OR3D1       7    17.640  tcbn65gplustc_ccs
OR3XD1      1     2.520  tcbn65gplustc_ccs
OR4D1       2     5.760  tcbn65gplustc_ccs
SDFKCNQD0    4    40.320  tcbn65gplustc_ccs
SDFQD0      1     8.640  tcbn65gplustc_ccs
SDFQND0     13   102.960  tcbn65gplustc_ccs
XNR2D1      77   277.200  tcbn65gplustc_ccs
XNR3D1     26   159.120  tcbn65gplustc_ccs
XNR4D0      2    18.000  tcbn65gplustc_ccs
XNR4D1      6    54.000  tcbn65gplustc_ccs
XOR2D1     107   385.200  tcbn65gplustc_ccs
XOR3D1     16    97.920  tcbn65gplustc_ccs
XOR4D1      4    36.000  tcbn65gplustc_ccs
-----
total        5237  22098.600

Type          Instances    Area    Area %
-----
sequential      965   8249.040   37.3
inverter        297   330.840    1.5
logic          3975  13518.720   61.2
physical_cells      0     0.000    0.0
-----
total          5237  22098.600  100.0

[EE671_36@vlsi73 Synthesis]$ |

```

Figure 12: Output of the gates.rpt

```

EE671_36@vlsi73:~/PnR_Exam X + | v
[EE671_36@vlsi73 Synthesis]$ cat power.rpt
Instance: /alu
Power Unit: W
PDB Frames: /stim#0/frame#0
-----
Category      Leakage      Internal      Switching      Total      Row%
-----
memory        0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
register      3.77579e-05  1.11670e-03  7.75876e-05  1.23204e-03  53.90%
latch         0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
logic         7.50511e-05  5.13700e-04  3.96328e-04  9.85079e-04  43.10%
bbox          0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
clock         0.00000e+00  0.00000e+00  6.87000e-05  6.87000e-05  3.01%
pad           0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
pm            0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
-----
Subtotal      1.12809e-04  1.63040e-03  5.42616e-04  2.28582e-03  100.01%
Percentage    4.94%       71.33%      23.74%      100.00% 100.00%
-----

[EE671_36@vlsi73 Synthesis]$ |

```

Figure 13: Output of the power.rpt

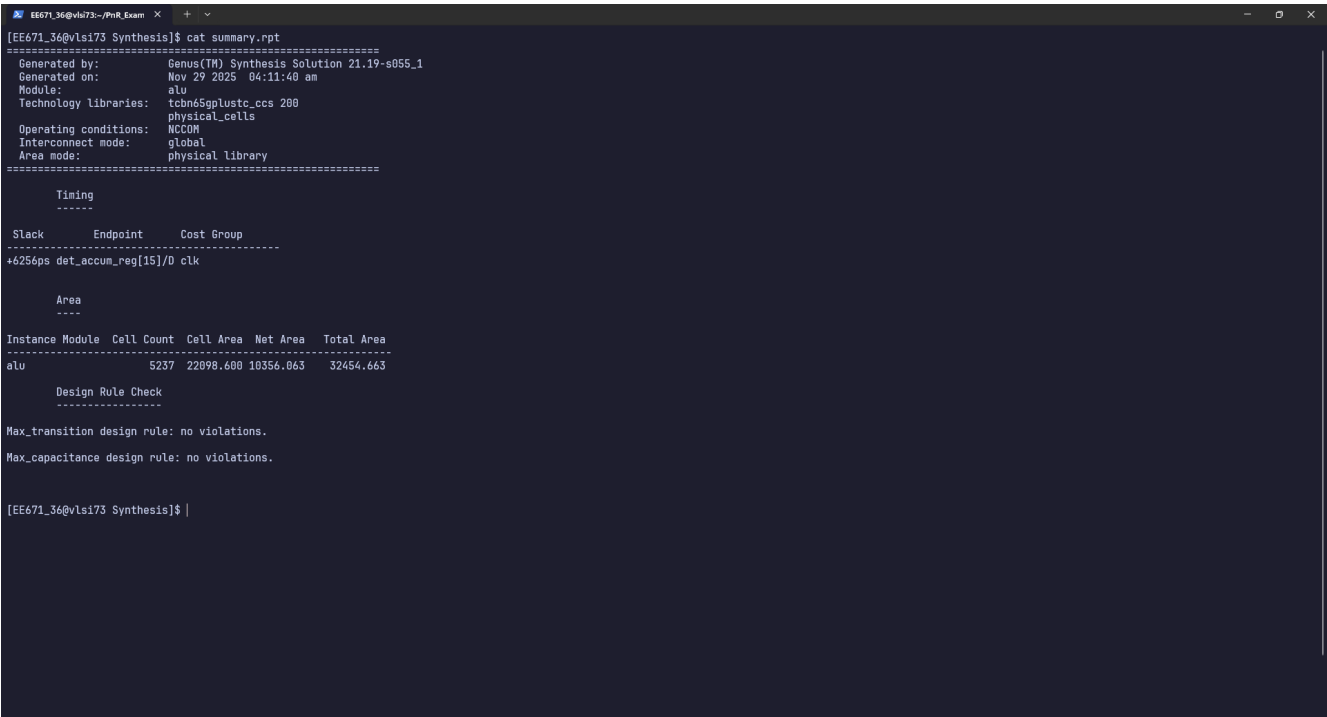


Figure 14: Output of the summary.rpt

4.1. GTKWave forms

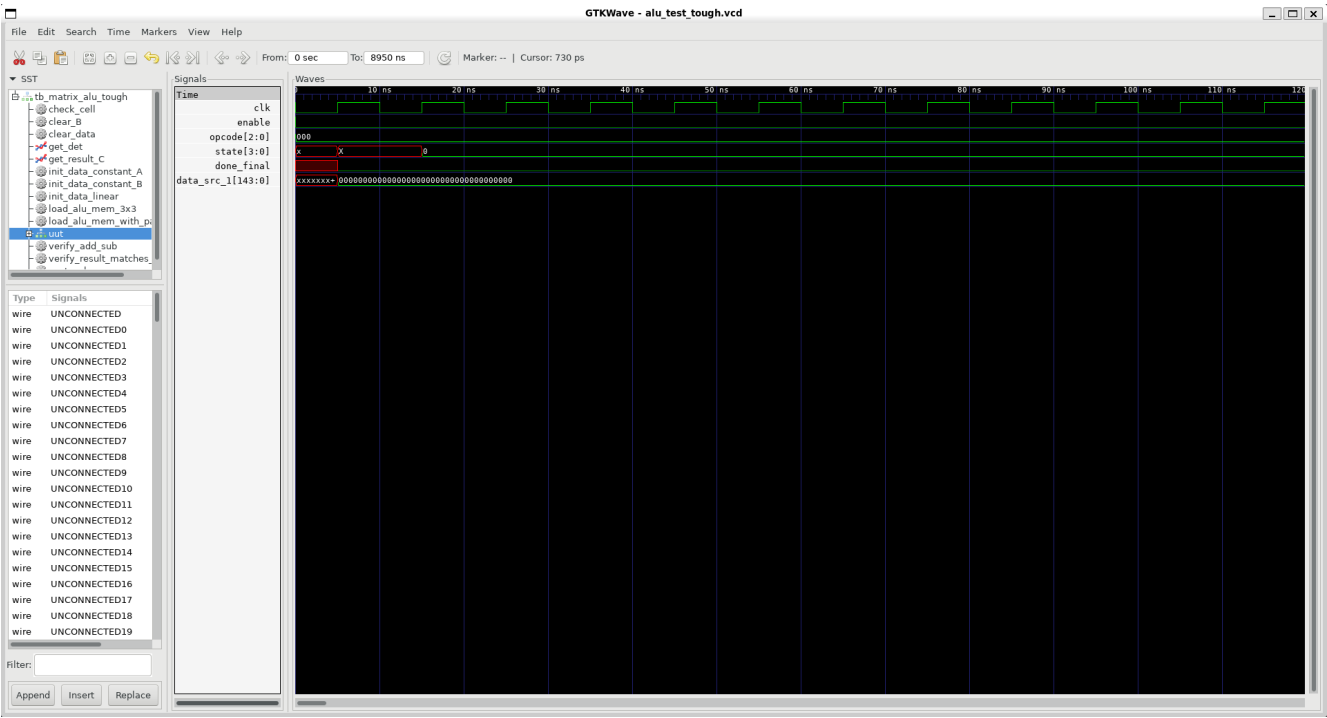
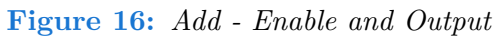


Figure 15: Taking Inputs



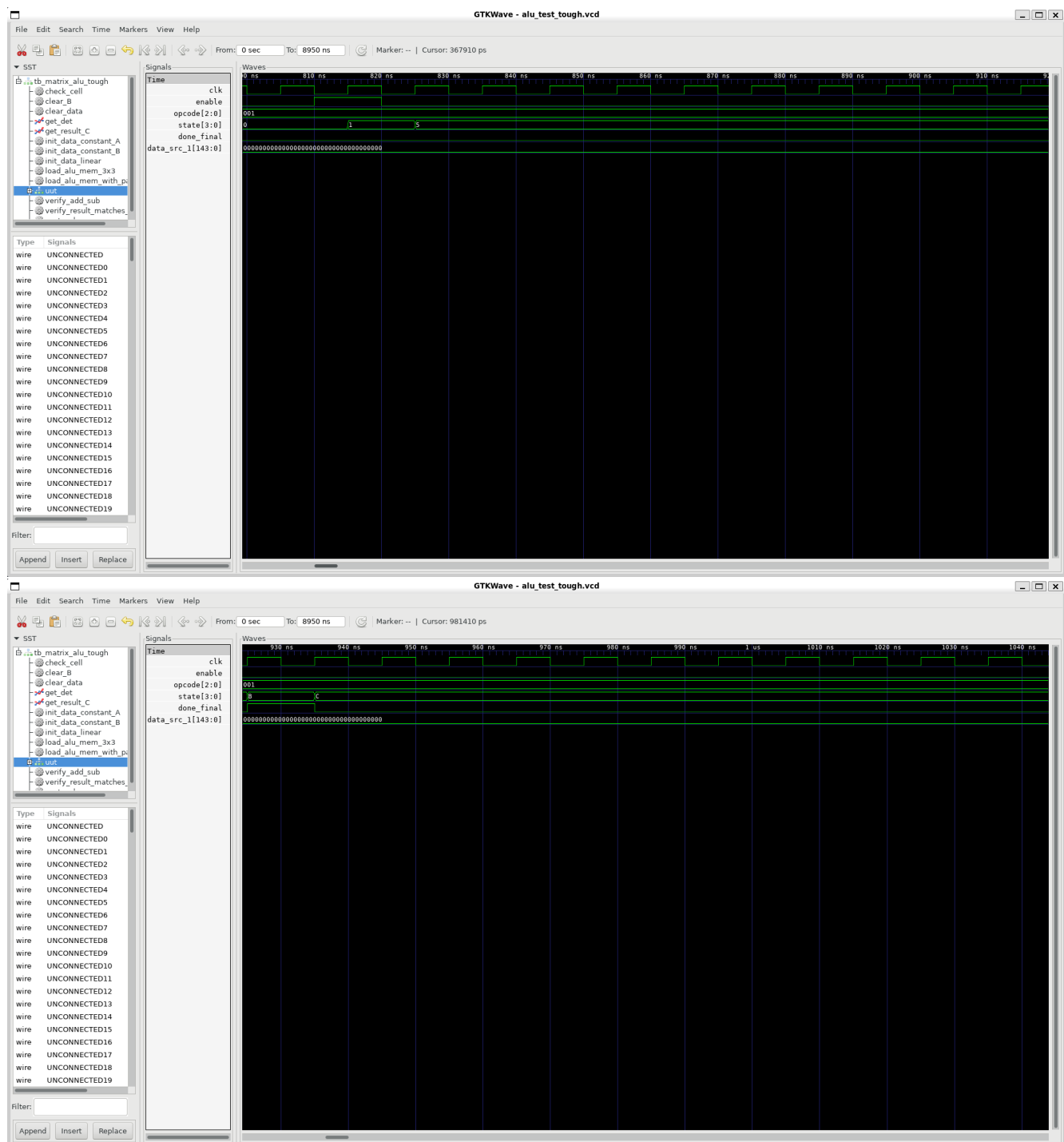
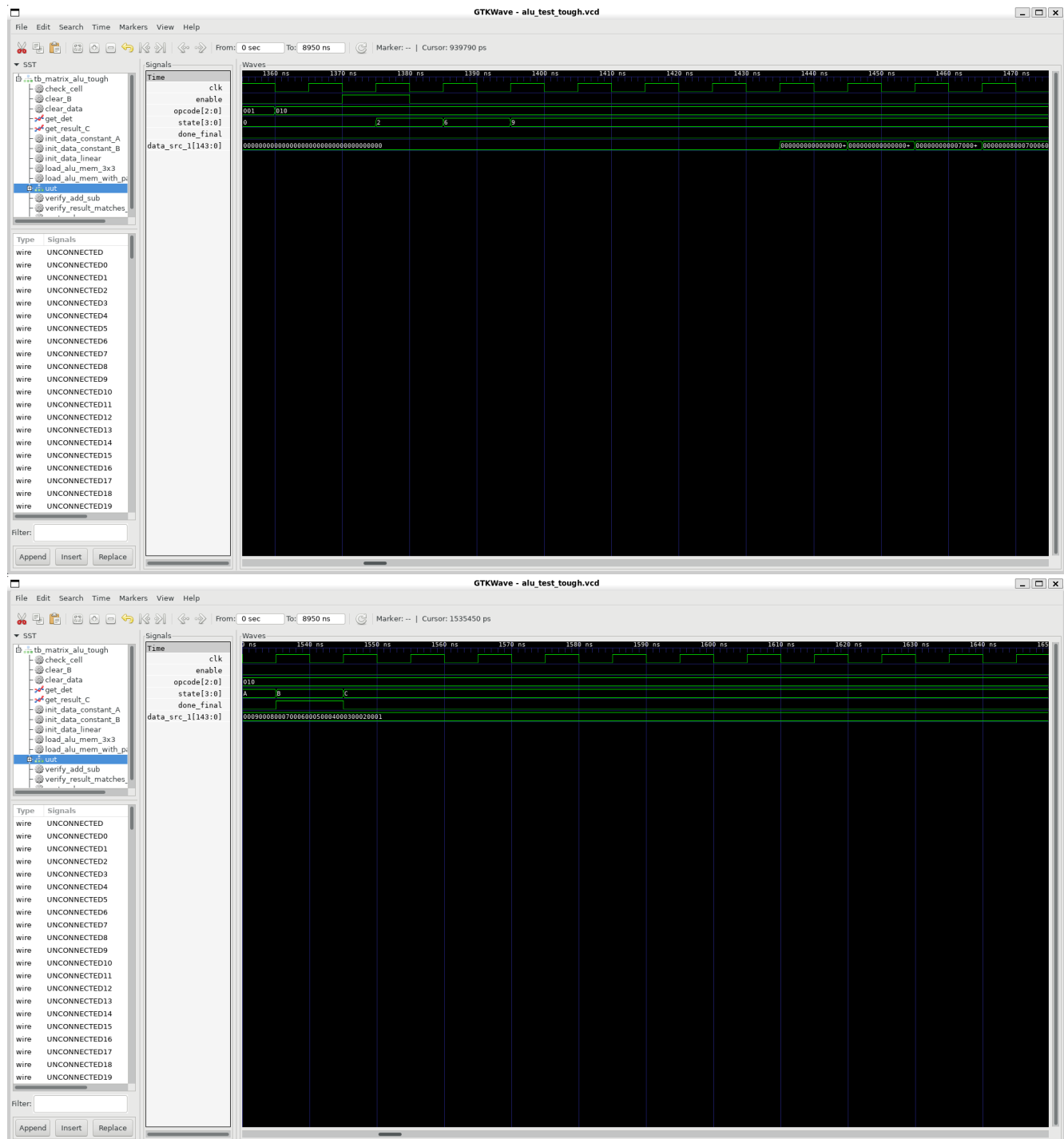


Figure 17: Sub - Enable and Output

Figure 18: *Mul - Enable and Output*

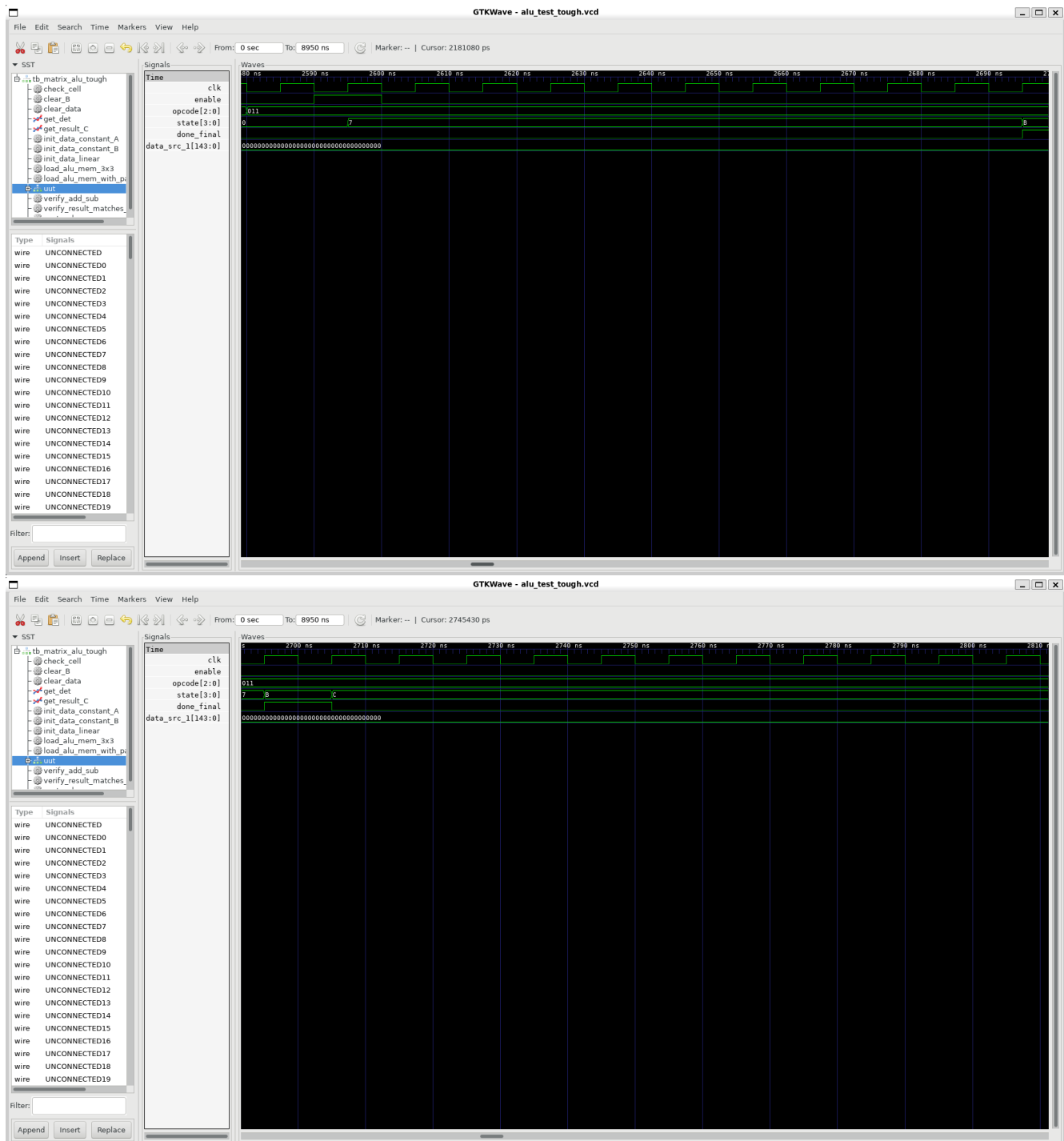


Figure 19: Transpose - Enable and Output

5. Physical Design

Now that we have the RTL generated, we can finally move on to the final step of this process, that being the Physical Design. After editing the pnr.tcl and mmmc.tcl file to set the basename and adding all of the pins we were all set to do the physical design. Initially, when the physical design is synthesized, we got a total of 4 DRC violations with no other violations in Antenna, Connectivity and in the Timing. To mitigate this, we used the Innovus tool command `ecoRoute` which helped us eliminate the DRC violations without introducing any other violations in Antenna, Connectivity or Timings. Please note that for the

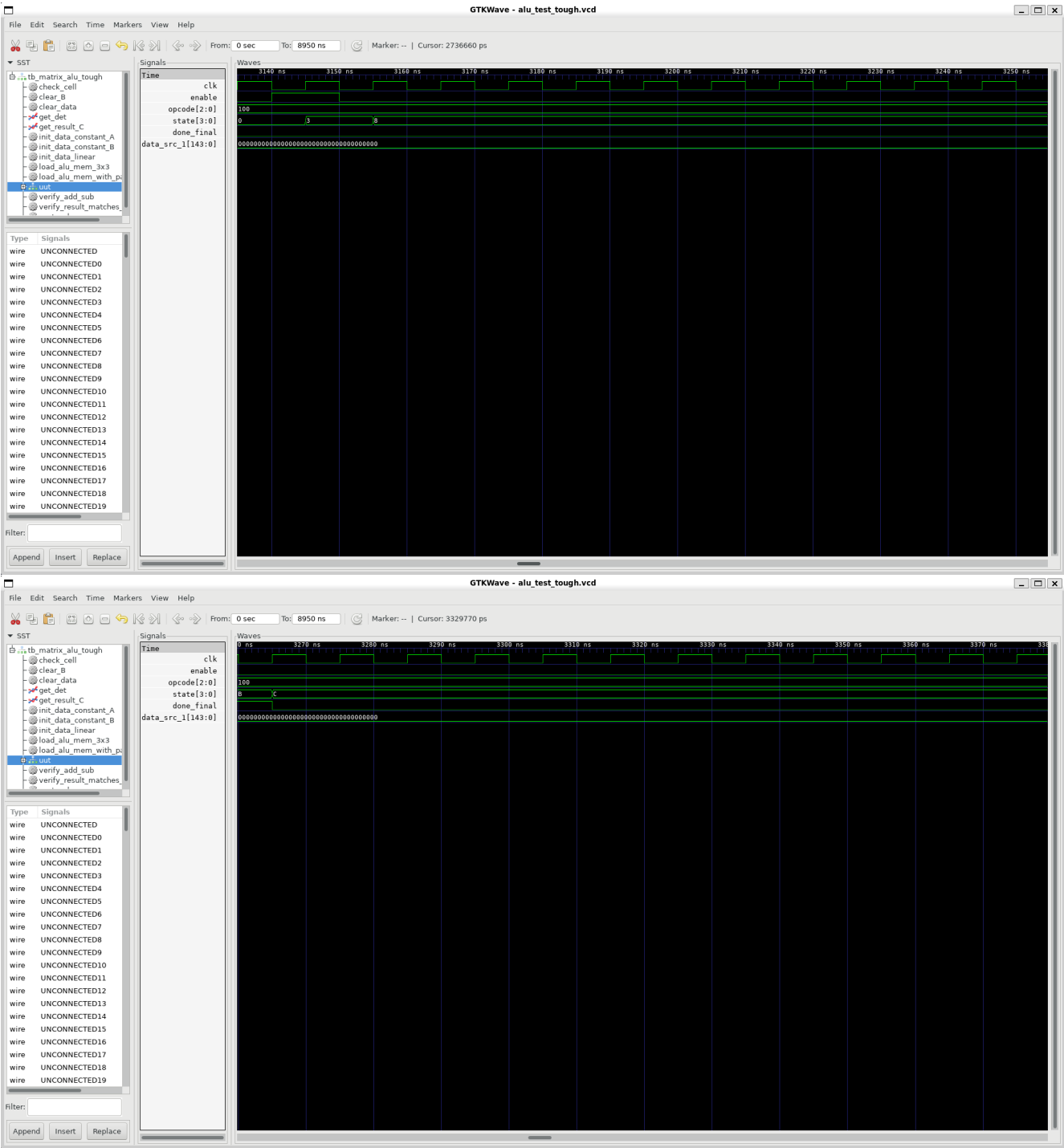


Figure 20: Inverse - Enable and Output

Physical Design, we made a new file called `pnr2.tcl`(explored in the Instructions to Replicate section of the report) and a line addition to the `innovus_PnR.sh`.

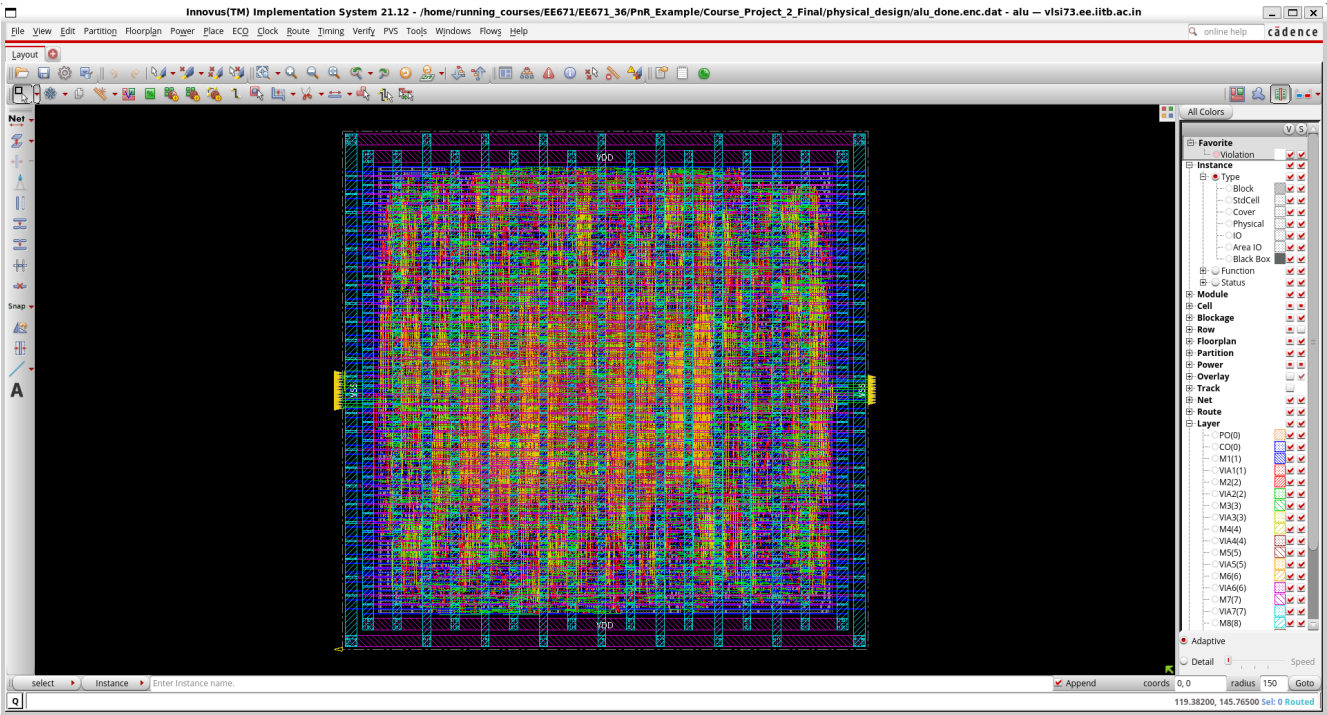


Figure 21: Physical Layout

5.1. GTKWave forms

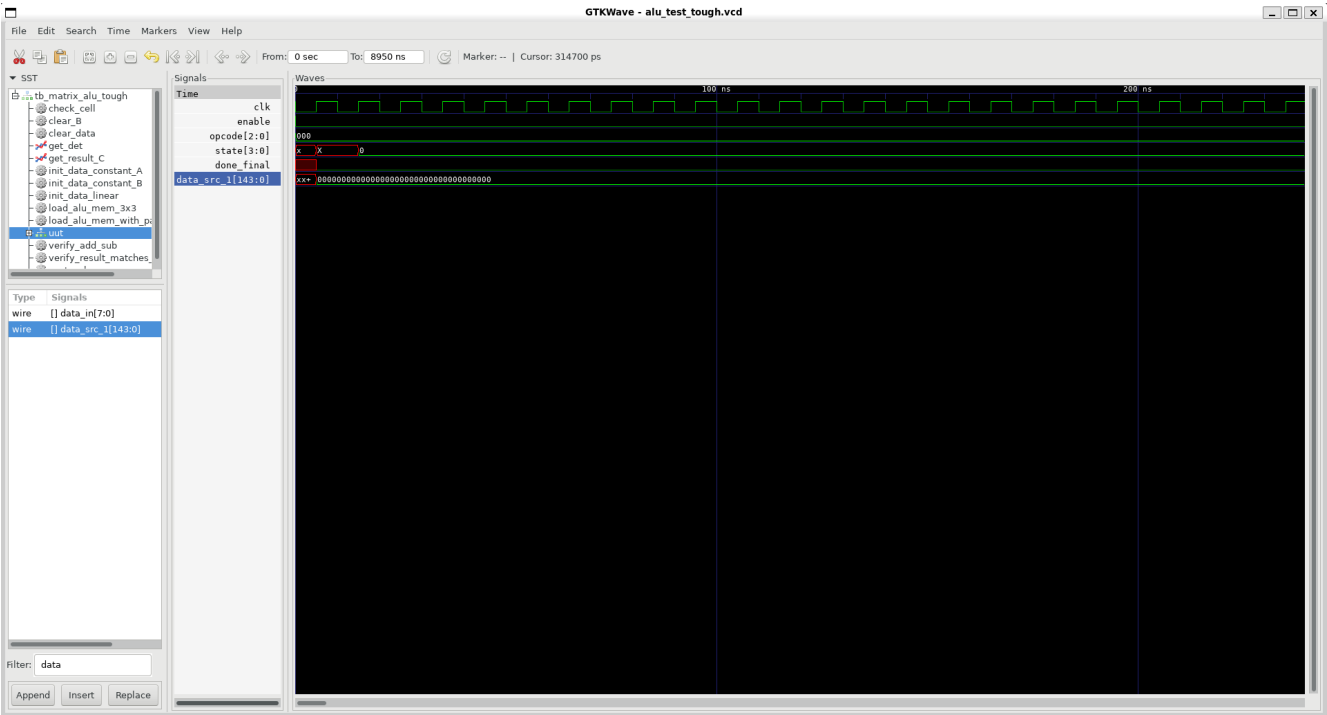


Figure 22: Taking Inputs

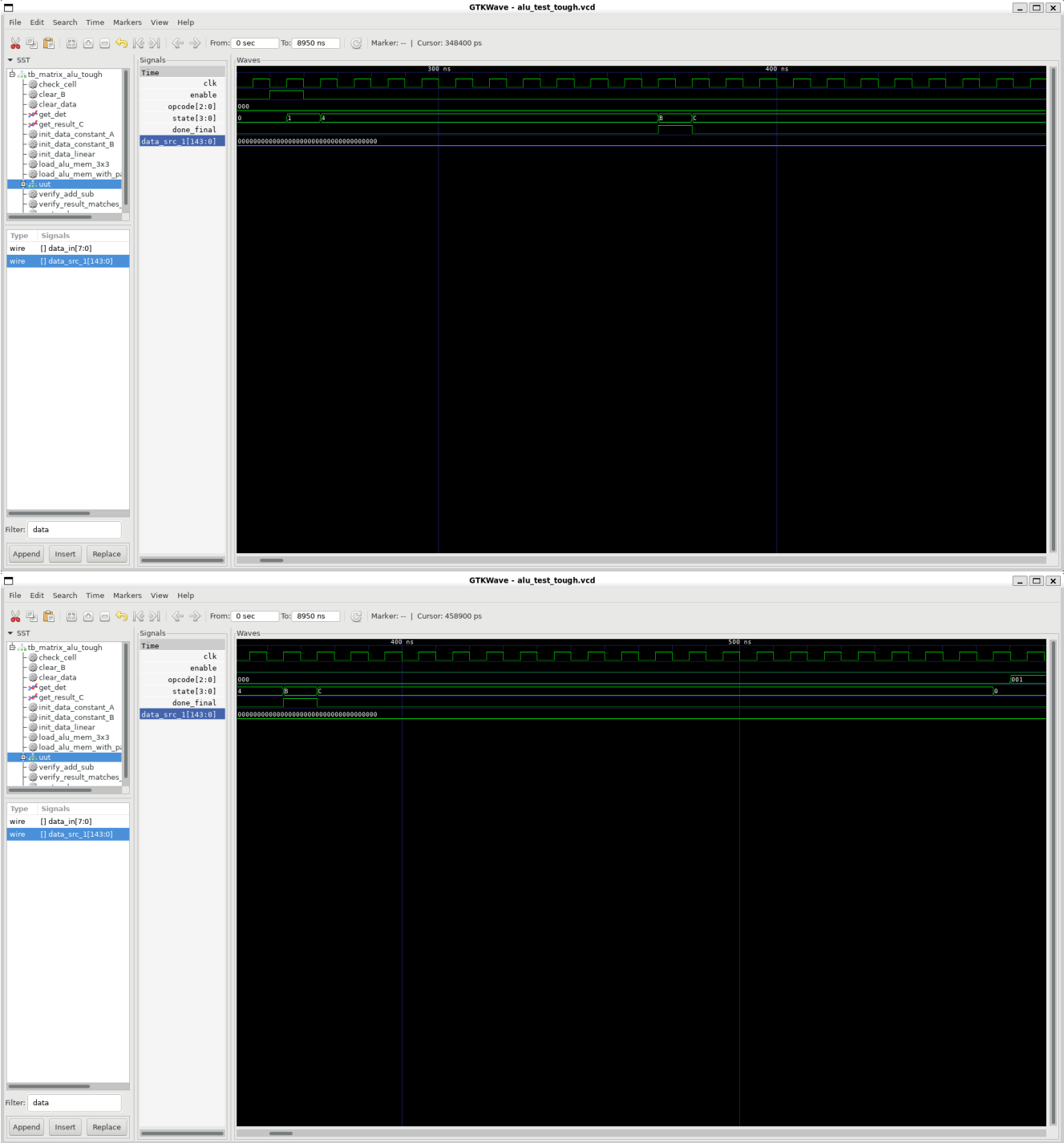


Figure 23: Add - Enable and Output

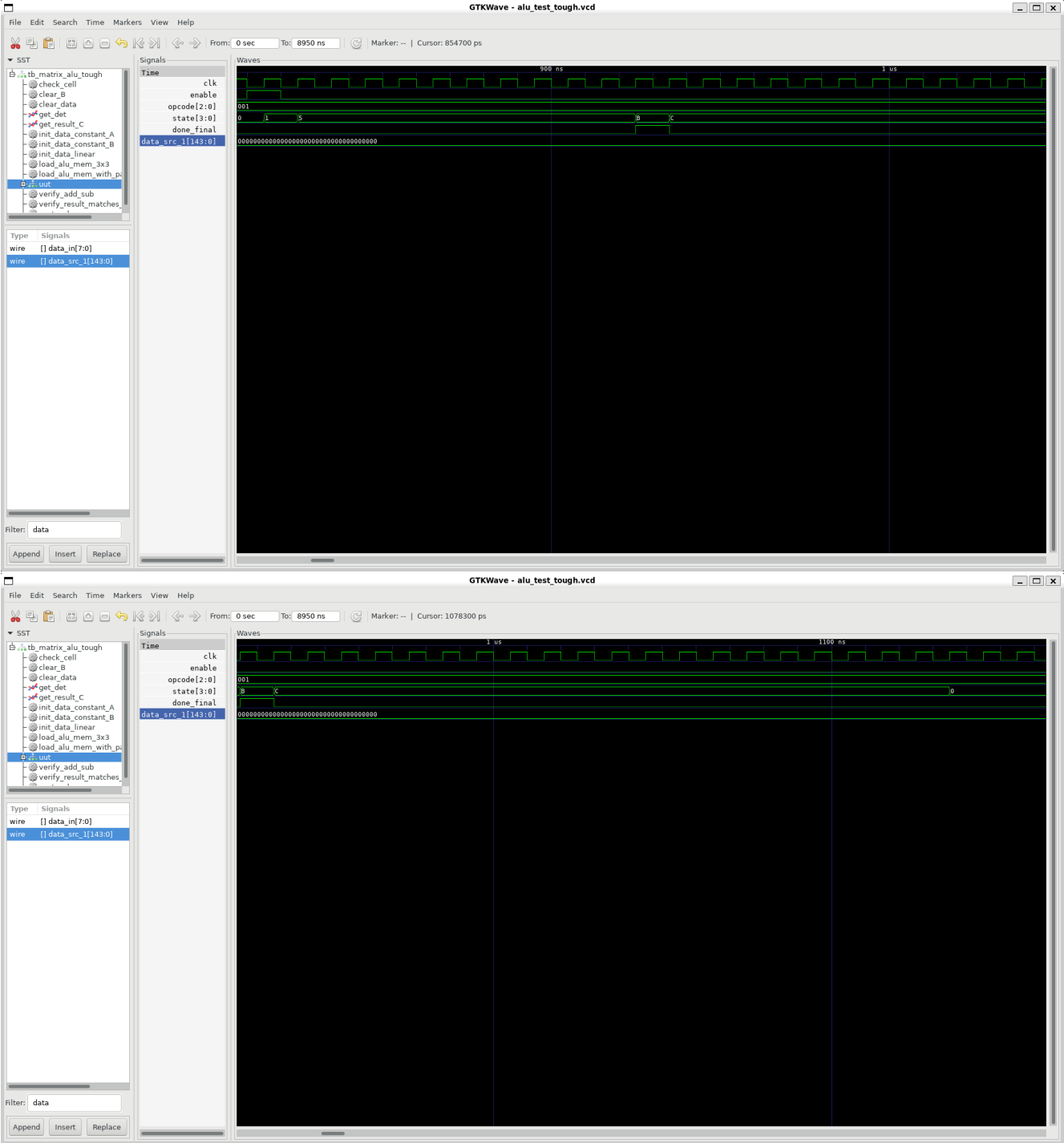


Figure 24: Sub - Enable and Output

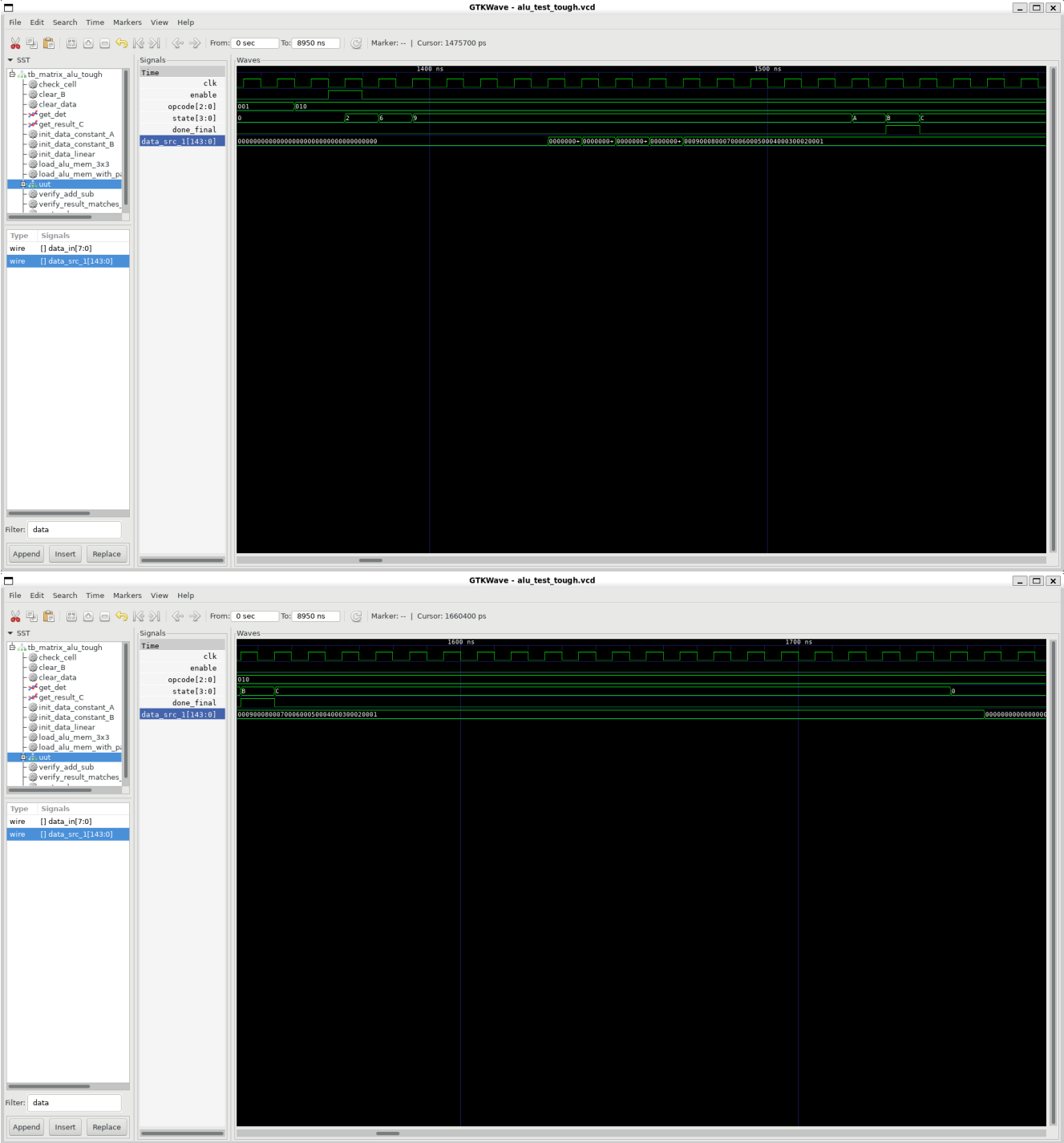
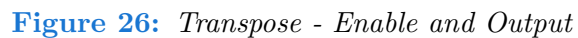
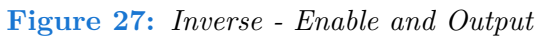


Figure 25: *Mul - Enable and Output*





5.2. Table of Reports

Table 1: *Table of Performance*

Metric	Value
Clock Frequency	100 MHz
Worst Case Setup Slack	4.433 ns
Worst Case Hold Slack	0.007 ns
Design Area	22305.24 μm^2
Internal Power	2.636 μW
Switching Power	0.9152 μW
Leakage Power	0.7432 μW
Total Power	4.294 μW

6. Instructions to Replicate

This section is to guide the user on how to go from the Verilog files to the Physical Design. The Verilog to RTL is largely unchanging, running the `genus_synthesis.sh` script would output the RTL file in the Verification directory. But there is a slight change to the RTL to GDSII part of the process, where we normally execute the Physical Process and automatically exiting and recall Innovus to now execute the elusive aforementioned `pnr2.tcl`. This tcl script was added late in the process and it starts with restoring the design and running the `ecoRoute` command before re-writing all of the netlists, exporting the DRC-free Design and the various timings reports. This change was made to ease the burden on the user in manually fixing the DRC violations and then generating all the reports.

7. Challenges faced

1. **Inability of tool to design the original promised design:** Initially, the plan was to use 32-bit integers and a maximum order of 4x4 for the ALU. However, upon testing a smaller version of the memory that would actually be required, we found that the tool was unable to synthesize said memory without throwing DRC violations. This led us to relax our specifications (as mentioned in a mail to the instructor and subsequently approved by him). This caused a significant delay and made us choose the current specifications of 8-bit integers and a maximum order of 3x3. Further, once we realized how to make DRC violations go away, it was too close to the submission deadline for us to make any changes to our architecture.
2. **2D memory synthesizing:** Upon resorting to the smaller memory, we tried to synthesize that with the ALU believing that it should work, but upon synthesis it was revealed that the synthesis tool wouldn't synthesize the memory unless an input pin drove it (this has been expanded upon in the I/O section).
3. **Matrix multiplier specific issue:** It was a difficult task to integrate the clocking requirements of the matrix multiplier with the ALU which was worsened by the complexity of blocking and non-blocking assignments. Even though the matrix multiplier was working by itself, coupling it with the ALU still took a long time and lots of debugging.
4. **Difficulty in fixing DRC violations:** Upon doing all of the above debugging and a successful synthesis, we got down to 4 DRC violations. This was subsequently resolved using the `ecoRoute` command manually after the `pnr.tcl` file had been executed. It was also required that the design and reports be saved manually after this step. Adding it directly in `pnr.tcl` didn't work, since

somehow a different behaviour was observed in the 2 cases: one, where directives were directly given in `pnr.tcl`, and the other in which they were input manually in a running innovus shell. This made it very hard to automate the process by using a single file, as we wanted it to be reproducible for evaluation and therefore be able to be run by a single command. Thus, we came up with `pnr2.tcl`, which would be run by `innovus_PnR.sh` after it was done with the first script. This has now successfully been automated, and everything now works with the single `./innovus_PnR.sh` shell command. The extra file, `pnr2.tcl` is elaborated upon in the "How to replicate" section.

5. **Disk space issues:** A few hours before the submission deadline, there were multiple cases of disk space being full on the common server. This cost us some delays as we hadn't backed up our project repository to our local machines and also delayed us from getting relevant screenshots from other windows like GTKWave and Innovus, due to the X11 forwarding breaking.

8. Contributions

Name	Contributions
Advay Bapat	• Constructed the entire matrix multiplication machinery, systolic entry modules, process elements. • Debugging the ALU, making the ALU testbench. • Determinant of 2x2 module. • Report making and proofreading of the Verilog design section.
Dinesh Methre	• Constructed the Kogge Stone adder–subtractor. • Debugging the ALU. • Physical design and synthesis of the ALU. • Report making and proofreading of the physical design section.
Pranav Malpani	• Constructed ALU state machine (adder–subtractor and FSM). • Debugging the ALU and ALU testbench. • Construction of the 8x8 Wallace–Booth multiplier and debugging of ALU. • Report making and proofreading of the Verilog design section.
Suketu Patni	• Constructed ALU state machine. • Inverse, determinant and transpose functionalities. • Physical design and synthesis of the ALU. • Report making and proofreading of the physical design, literature review and inverse section.